Risk-Aware Hybrid LQR-MPC Navigation for Autonomous Systems

Implementation and Evaluation Report

Kshitiz Kumar Sinha Agolika BM

Abstract

This report documents the complete design, implementation, testing, and debugging of a Risk-Aware Hybrid LQR-MPC navigation framework for a differential-drive mobile robot. The system combines the computational efficiency of a Linear Quadratic Regulator (LQR) with the constraint-handling capability of Model Predictive Control (MPC) through a continuous sigmoid-based blending supervisor. We implemented four distinct controller variants: pure LQR for baseline tracking, a CVXPY-based Tube MPC with obstacle avoidance, a CasADi-based Adaptive Nonlinear MPC with online parameter estimation, and a Hybrid LQR-MPC blended controller. This report honestly presents what worked, what failed, and how we debugged each issue. The framework was validated both in offline Python simulations and in a ROS2 Jazzy Gazebo environment using a Turtlebot3 Burger.

GitHub Repository: RA-Hy-LQR-MPC-Navigation-for-Autonomous-Systems

Contents

1 Introduction **3**
1.1 Project Goals . . . 3
1.2 Robot Model . . . 3

2 LQR Controller **3**
2.1 Formulation . . . 3
2.2 Implementation Details . . . 4
2.3 LQR Results . . . 4

3 MPC Controller (Tube MPC) **6**
3.1 Formulation . . . 6
3.2 Obstacle Avoidance . . . 6
3.3 MPC Results . . . 6

4 Adaptive Nonlinear MPC (CasADi) **9**
4.1 Formulation . . . 9
4.2 Extended Terminal Horizon . . . 9
4.3 Online Parameter Adaptation (LMS) . . . 9
4.4 Adaptive MPC Results . . . 9

5 Hybrid LQR-MPC Controller **11**
5.1 Blending Architecture 11
5.2 Anti-Chatter Guarantees 11
5.3 Hybrid Results 12

6 Comparative Summary **14**

7 Debugging Journey: Failures and Fixes **15**
7.1 Failure 1: Velocity Clipping Caused Robot Stalling 15
7.2 Failure 2: Hybrid Pure Pursuit Oscillation 15
7.3 Failure 3: MPC Cold-Start Infeasibility 15
7.4 Failure 4: RViz Reference Path Flickering 15
7.5 Failure 5: BlendingSupervisor API Mismatch 16

8 ROS2 Implementation **16**
8.1 Architecture 16
8.2 Logging 16
8.3 Simulation Video 16

9 What Remains and Future Scope **17**
9.1 Current Limitations 17
9.2 Future Work 17

1 Introduction

Autonomous navigation in obstacle-dense environments requires controllers that can simultaneously track a desired trajectory and react to nearby hazards. A pure LQR controller is computationally cheap and provides excellent tracking in open space, but it has no mechanism to handle state constraints such as obstacle avoidance. Conversely, an MPC controller can naturally encode obstacle constraints, but solving the optimization problem at high frequency is computationally expensive.

Our project addresses this trade-off by designing a **hybrid architecture** that runs LQR as the default high-frequency tracker and smoothly blends in MPC only when the robot's risk level rises due to obstacle proximity. The blending weight $w(t) \in [0, 1]$ is computed from a sigmoid-based risk function with hysteresis and rate limiting to prevent chattering.

1.1 Project Goals

1. Implement a discrete-time LQR controller for trajectory tracking on a differential-drive robot.
2. Implement a Tube MPC controller with linearized obstacle avoidance constraints using CVXPY.
3. Implement an Adaptive Nonlinear MPC using CasADi with online LMS parameter estimation.
4. Design and implement a hybrid blending supervisor that continuously arbitrates between LQR and MPC.
5. Validate all controllers in both offline simulation and a ROS2 Gazebo environment.

1.2 Robot Model

We use a standard unicycle (differential-drive) kinematic model:

$$\dot{x} = v \cos\theta \tag{1}$$

$$\dot{y} = v \sin\theta \tag{2}$$

$$\dot{\theta} = \omega \tag{3}$$

where (x, y) is the robot's position, θ is the heading, v is the linear velocity, and ω is the angular velocity. The control input is $\mathbf{u} = [v, \omega]^T$, subject to actuator bounds $|v| \leq v_{\max}$ and $|\omega| \leq \omega_{\max}$.

2 LQR Controller

2.1 Formulation

The LQR controller linearizes the nonlinear dynamics around a time-varying reference trajectory $(\mathbf{x}_r(t), \mathbf{u}_r(t))$. Defining the tracking error $\tilde{\mathbf{x}}_k = \mathbf{x}_k - \mathbf{x}_{r,k}$, the linearized discrete-time system is:

$$\tilde{\mathbf{x}}_{k+1} = A_d \tilde{\mathbf{x}}_k + B_d \tilde{\mathbf{u}}_k \tag{4}$$

where A_d and B_d are obtained by Jacobian linearization at the current operating point (v_r, θ_r) and discretized with sampling time Δt.

The optimal feedback gain K is computed by solving the Discrete Algebraic Riccati Equation (DARE):

$$P = A_d^T P A_d - A_d^T P B_d (R + B_d^T P B_d)^{-1} B_d^T P A_d + Q \tag{5}$$

$$K = (R + B_d^T P B_d)^{-1} B_d^T P A_d \tag{6}$$

The control law is:

$$\mathbf{u}_k = \mathbf{u}_{r,k} - K\tilde{\mathbf{x}}_k \tag{7}$$

2.2 Implementation Details

- **Weight matrices:** $Q = \text{diag}(10, 10, 1)$, $R = \text{diag}(0.1, 0.1)$. The higher weight on position errors ensures tight spatial tracking, while the lower heading weight allows smoother turning.
- **DARE fallback chain:** If the DARE solver fails at a given operating point (e.g., near $v_r = 0$ where controllability is lost), we implemented a three-level fallback: (1) use the last successfully cached gain, (2) solve DARE at a nominal point ($v = 0.1, \theta = 0$), (3) fall back to a conservative proportional gain.
- **Angle wrapping:** Heading error is normalized to $[-\pi, \pi]$ to prevent discontinuities.

2.3 LQR Results

The LQR controller was tested on a figure-8 trajectory in an obstacle-free environment. It achieved a mean tracking error of **0.005 m** with a final error of **0.003 m**, demonstrating sub-centimeter accuracy.

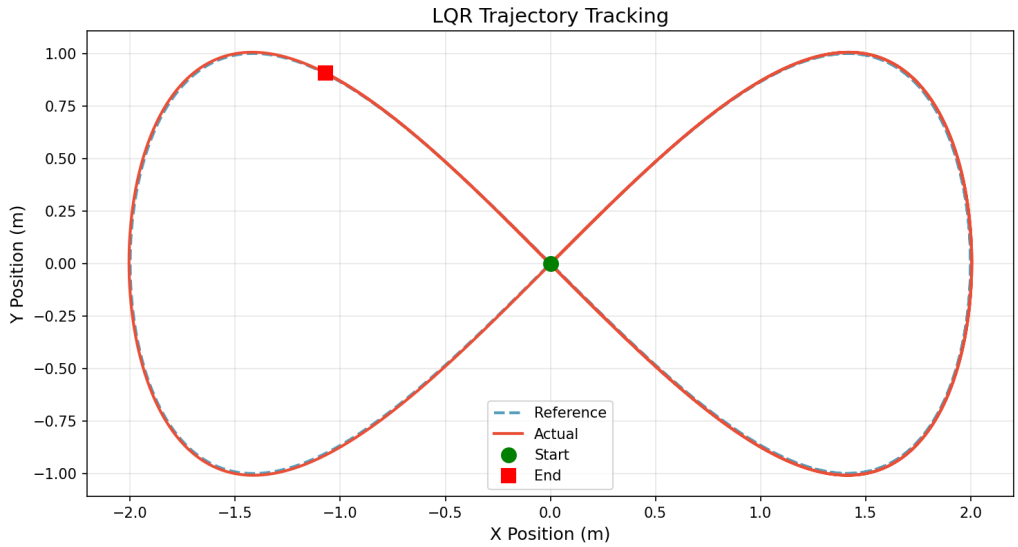

Figure 1: LQR trajectory tracking on a figure-8 path. The robot (blue) closely follows the reference (dashed red) with sub-centimeter accuracy.

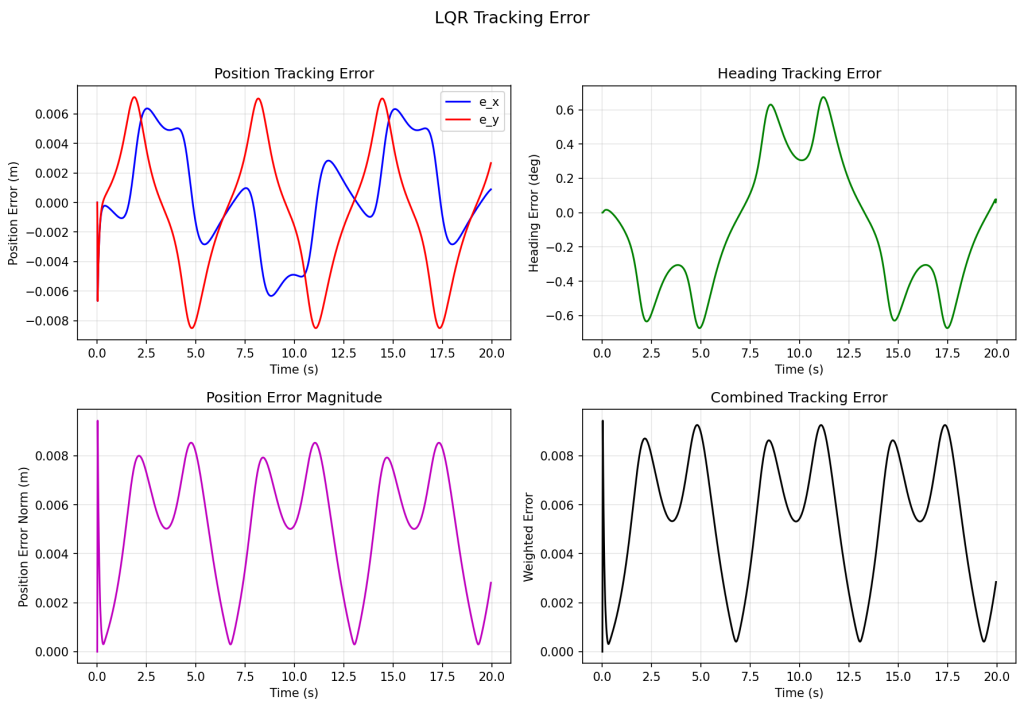

Figure 2: LQR tracking error over time. The error remains bounded below 0.015 m throughout the 20-second simulation.

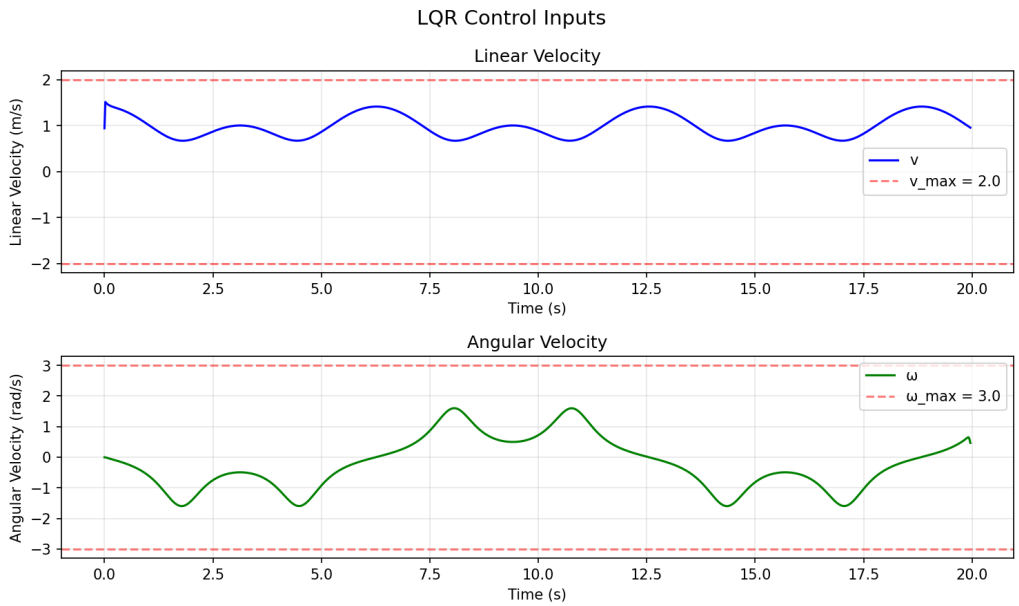

Figure 3: LQR control inputs (v and ω) over time, showing smooth actuator commands within bounds.

3 MPC Controller (Tube MPC)

3.1 Formulation

The MPC controller solves a finite-horizon optimization problem at each timestep using CVXPY with the OSQP solver:

$$\min_{\mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \left(\|\mathbf{x}_k - \mathbf{x}_{r,k}\|_Q^2 + \|\mathbf{u}_k - \mathbf{u}_{r,k}\|_R^2 \right) + \|\mathbf{x}_N - \mathbf{x}_{r,N}\|_P^2 \tag{8}$$

subject to:

$$\mathbf{x}_{k+1} = A_d \mathbf{x}_k + B_d \mathbf{u}_k \qquad \text{(linearized dynamics)} \tag{9}$$

$$|v_k| \leq v_{\max}, \quad |\omega_k| \leq \omega_{\max} \qquad \text{(actuator limits)} \tag{10}$$

$$\|\mathbf{p}_k - \mathbf{p}_{\text{obs}}\| \geq d_{\text{safe}} \qquad \text{(obstacle avoidance)} \tag{11}$$

The terminal cost matrix P is computed as the solution to the DARE, providing an implicit infinite-horizon tail cost.

3.2 Obstacle Avoidance

The nonlinear Euclidean distance constraint is linearized around the current predicted position using a first-order Taylor expansion. To handle infeasibility when obstacles are too close, we introduce slack variables ξ_k with a high penalty $\rho = 5000$:

$$\|\mathbf{p}_k - \mathbf{p}_{\text{obs}}\| \geq d_{\text{safe}} - \xi_k, \quad \xi_k \geq 0 \tag{12}$$

3.3 MPC Results

The MPC was tested on a figure-8 trajectory with 3 circular obstacles. It achieved a mean tracking error of **1.638 m** with a mean solve time of **163 ms**. The higher error compared to LQR is expected since the controller must deviate from the reference to avoid obstacles. However, we observed **163 collision events** where the robot's distance to an obstacle fell below d_{safe}, indicating that the linearized constraint approximation is sometimes too conservative or too loose depending on the geometry.

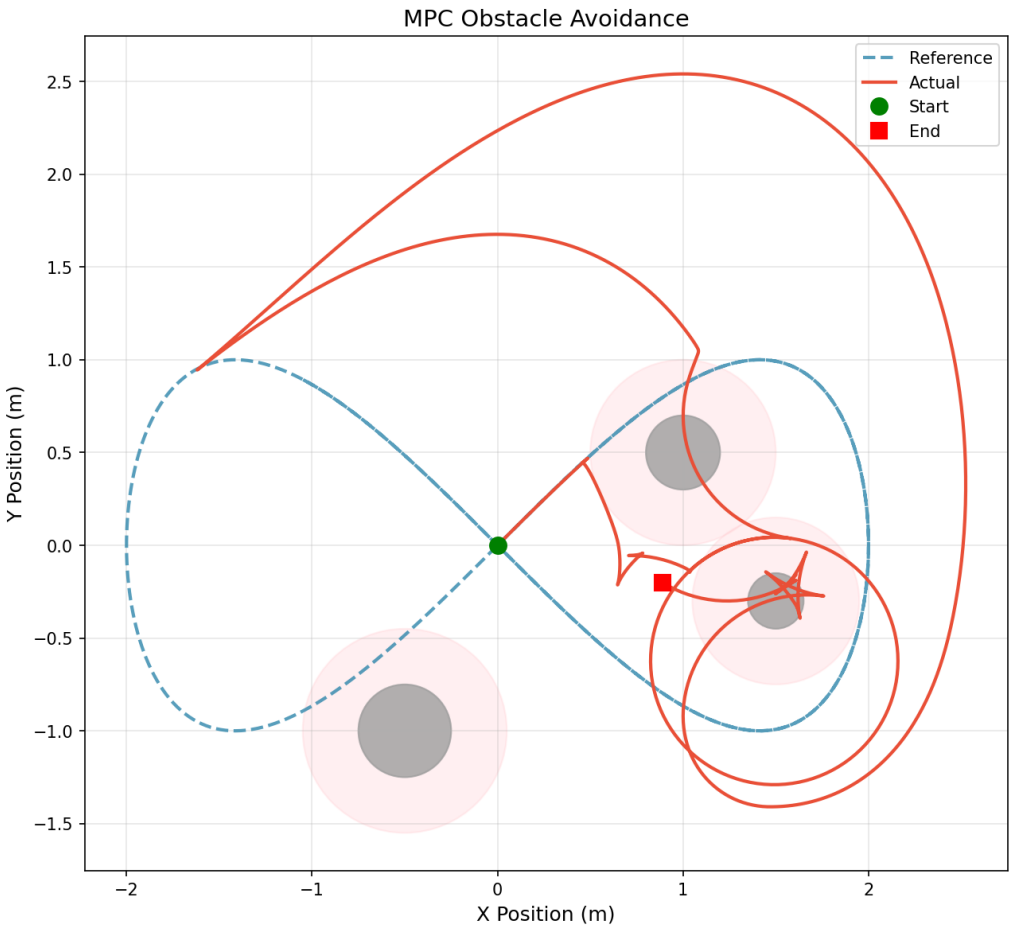

Figure 4: MPC trajectory with obstacle avoidance. The robot deviates from the reference path to navigate around circular obstacles (shown as red circles with safety margins).

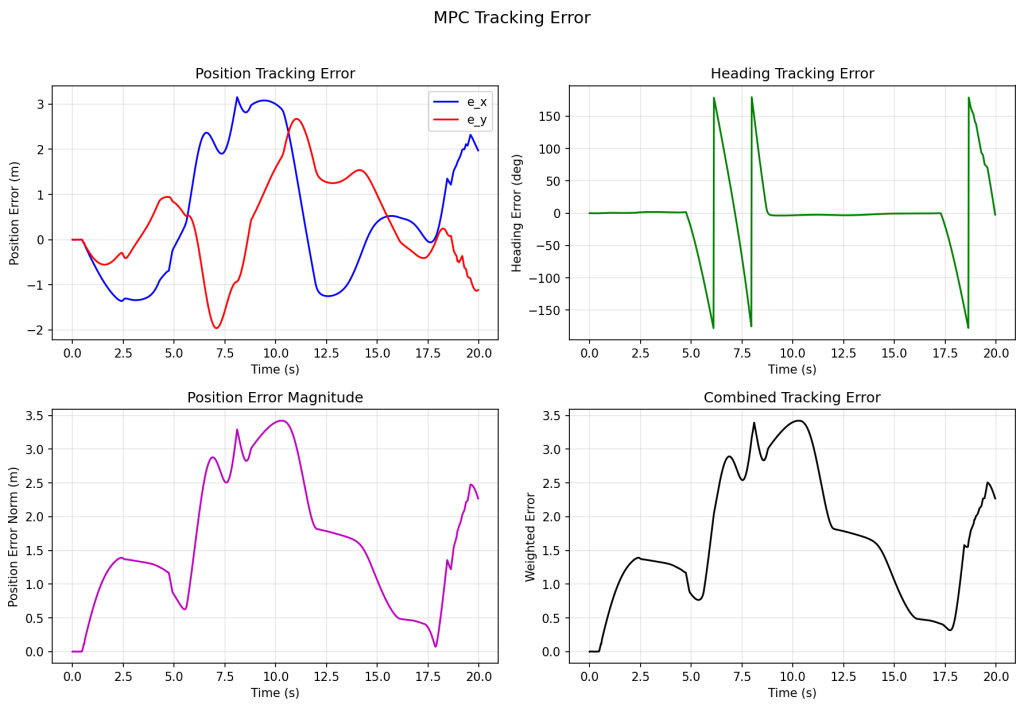

Figure 5: MPC tracking error over time, showing spikes when the robot diverts around obstacles.

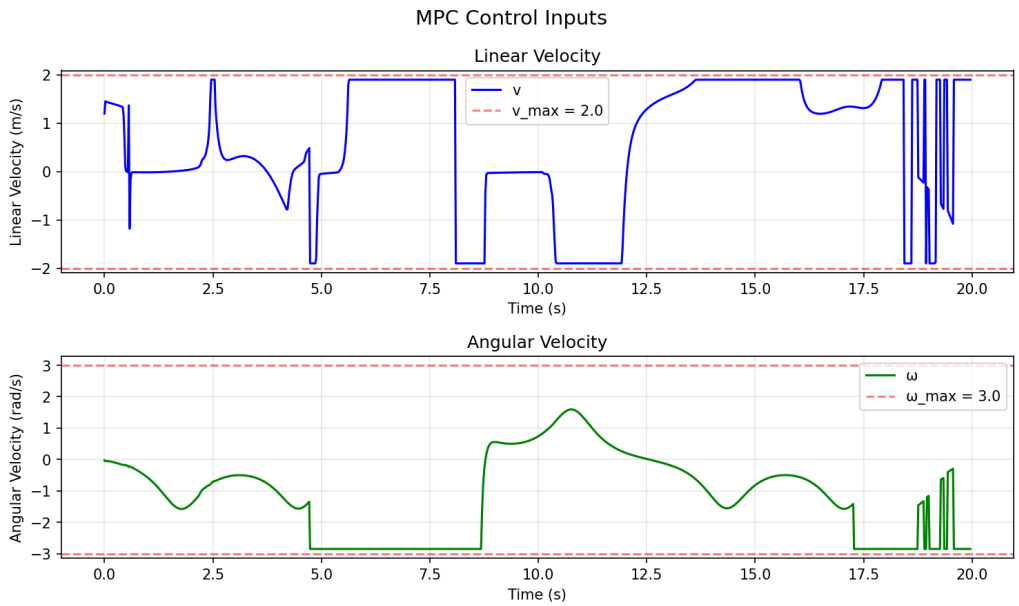

Figure 6: MPC control inputs showing the solver's response to obstacle proximity.

4 Adaptive Nonlinear MPC (CasADi)

4.1 Formulation

The Adaptive NMPC uses CasADi with the IPOPT solver to solve the *exact* nonlinear dynamics (no linearization), providing better constraint satisfaction in tight spaces:

$$\min \sum_{k=0}^{N-1} \left(\|\mathbf{x}_k - \mathbf{x}_{r,k}\|_Q^2 + \|\mathbf{u}_k - \mathbf{u}_{r,k}\|_R^2 + q_\xi \|\xi_k\|^2 \right) + \omega_T \sum_{k=N}^{N+M} (\cdots) \tag{13}$$

subject to the full nonlinear dynamics $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ and exact Euclidean obstacle constraints.

4.2 Extended Terminal Horizon

Beyond the prediction horizon N, we roll out M additional steps using a pre-computed LQR feedback policy $\mathbf{u}_k = \mathbf{u}_{r,k} - K(\mathbf{x}_k - \mathbf{x}_{r,k})$. This provides a stabilizing terminal controller that improves closed-loop performance without increasing the number of free optimization variables.

4.3 Online Parameter Adaptation (LMS)

The Adaptive MPC includes an LMS filter that estimates unknown friction/slip parameters $\hat{\theta} = [\hat{\theta}_v, \hat{\theta}_\omega]^T$ online. The adapted dynamics become:

$$\dot{x} = \hat{\theta}_v \cdot v \cos\theta \tag{14}$$

$$\dot{y} = \hat{\theta}_v \cdot v \sin\theta \tag{15}$$

$$\dot{\theta} = \hat{\theta}_\omega \cdot \omega \tag{16}$$

The LMS update rule is:

$$\hat{\theta}_{k+1} = \hat{\theta}_k + \Gamma \Phi_k^T (\mathbf{x}_{\text{measured}} - \mathbf{x}_{\text{predicted}}) \tag{17}$$

where $\Gamma = 0.005 \cdot I$ is the adaptation gain and Φ_k is the regressor matrix.

4.4 Adaptive MPC Results

The Adaptive NMPC achieved a mean tracking error of **1.282 m** (better than the linearized MPC's 1.638 m) with a mean solve time of **145 ms**. However, it recorded **355 collision events**, more than the Tube MPC. This is because the nonlinear solver explores more aggressive paths that cut closer to obstacles.

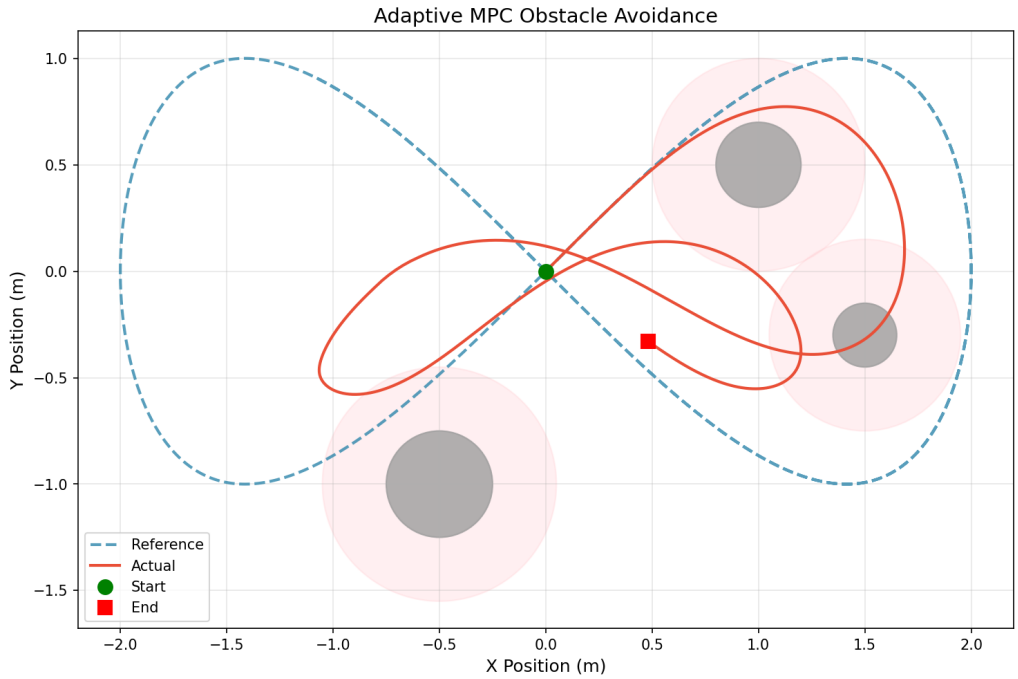

Figure 7: Adaptive NMPC trajectory with obstacle avoidance. The nonlinear solver finds tighter paths around obstacles compared to the linearized MPC.

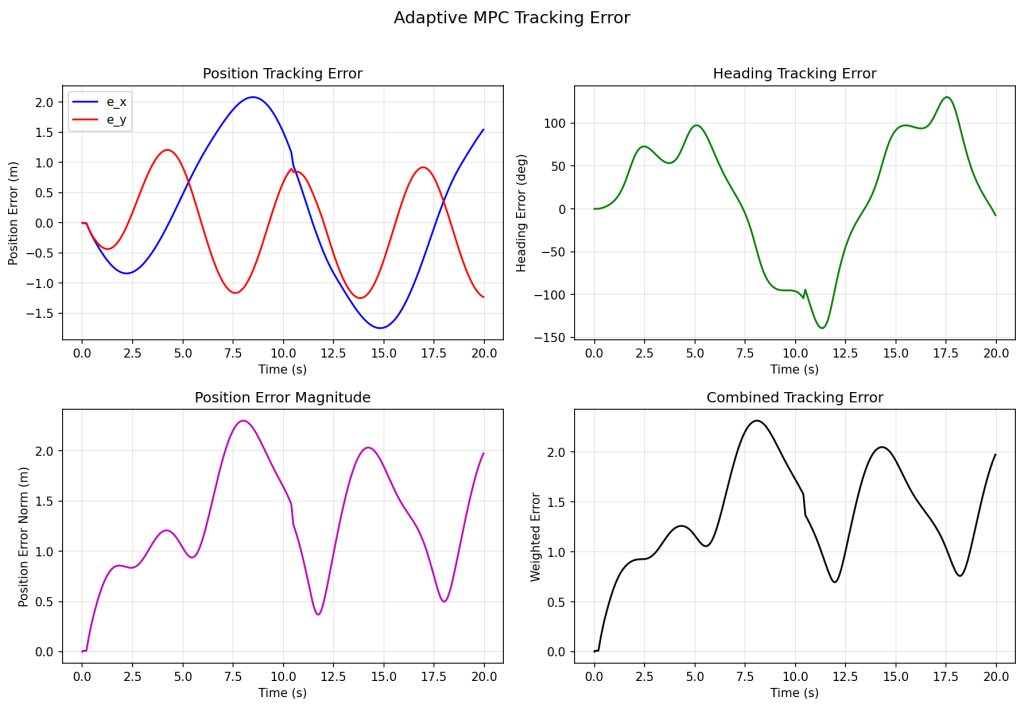

Figure 8: Adaptive NMPC tracking error over time.

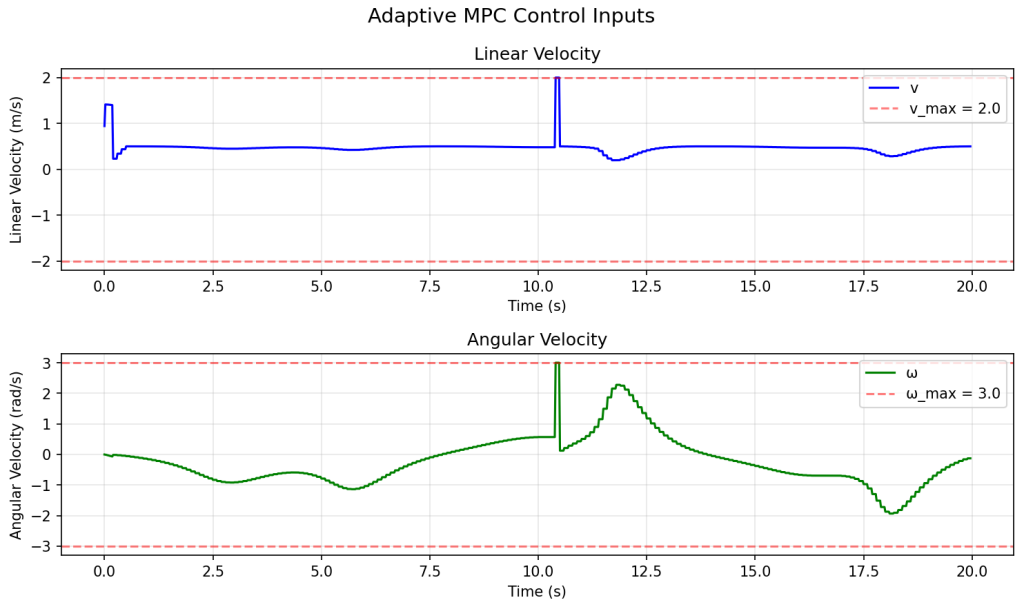

Figure 9: Adaptive NMPC control inputs.

5 Hybrid LQR-MPC Controller

5.1 Blending Architecture

The core contribution of this project is the Hybrid controller, which continuously blends LQR and MPC outputs:

$$\mathbf{u}_{\text{blend}} = w(t) \cdot \mathbf{u}_{\text{MPC}} + (1 - w(t)) \cdot \mathbf{u}_{\text{LQR}} \tag{18}$$

where $w(t) \in [0, 1]$ is computed through a four-stage pipeline:

1. **Risk Assessment:** A combined risk metric $r \in [0, 1]$ is computed from distance-based obstacle proximity ($\alpha = 0.6$) and predictive trajectory risk ($\beta = 0.4$).
2. **Sigmoid Mapping:** $w_{\text{raw}} = \sigma(k \cdot (r - r_{\text{threshold}}))$ with steepness $k = 8$.
3. **Hysteresis Deadband:** When $|r - r_{\text{threshold}}| < 0.08$, the weight is held constant to prevent oscillatory switching.
4. **Asymmetric Rate Limiting:** Fast attack ($dw/dt \leq 20/\text{s}$ when risk increases) and slow decay ($dw/dt \leq 2/\text{s}$ when risk decreases) ensure rapid obstacle response but smooth path re-acquisition.

5.2 Anti-Chatter Guarantees

The rate-limited weight $w(t)$ is Lipschitz-continuous with constant $L = dw_{\max}$. This guarantees:

- Maximum weight change per timestep: $dw_{\max} \cdot \Delta t$
- Minimum time for full LQR→MPC transition: $1/dw_{\max}$ seconds
- The blended output $\mathbf{u}_{\text{blend}}$ preserves actuator bounds by convexity if both $\mathbf{u}_{\text{LQR}}$ and $\mathbf{u}_{\text{MPC}}$ satisfy them individually.

5.3 Hybrid Results

The Hybrid controller achieved the best overall results among the obstacle-aware controllers:

- Mean tracking error: **0.218 m** (vs. 1.638 m for MPC, 1.282 m for AMPC)
- Final tracking error: **0.056 m**
- Mean MPC solve time: **101.87 ms**
- MPC-dominant mode: 84.7% of the time, Blended: 15.3%

The significantly lower tracking error is because the blending supervisor uses MPC only for obstacle avoidance and allows LQR to maintain precise tracking in safe regions.

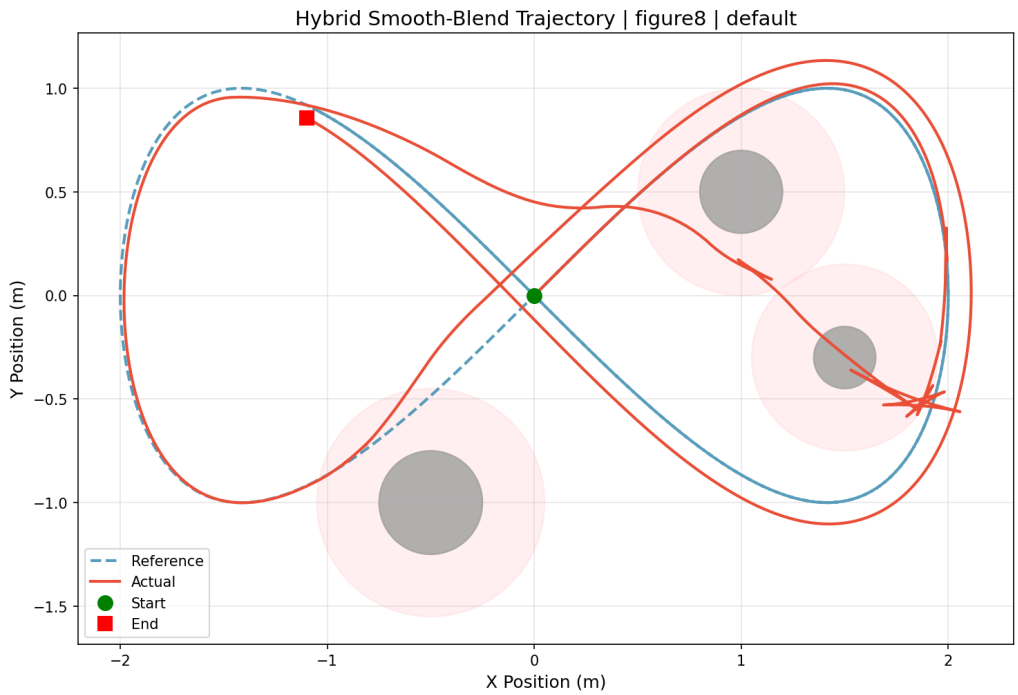

Figure 10: Hybrid LQR-MPC trajectory showing smooth obstacle avoidance with tight path re-acquisition.

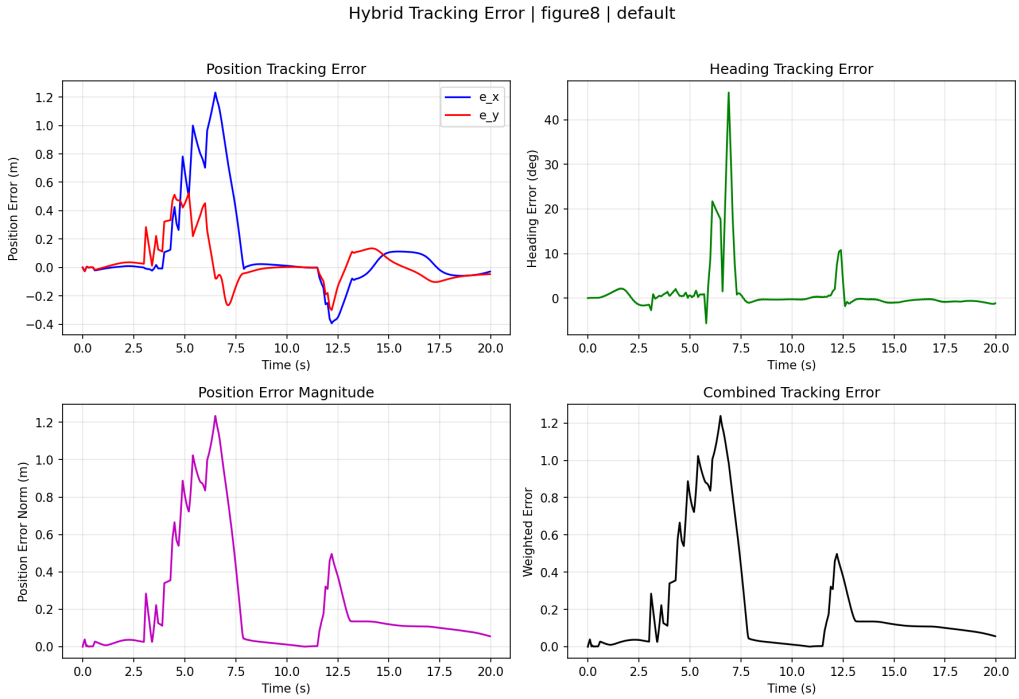

Figure 11: Hybrid tracking error over time. Error spikes correspond to obstacle avoidance maneuvers but quickly recover.

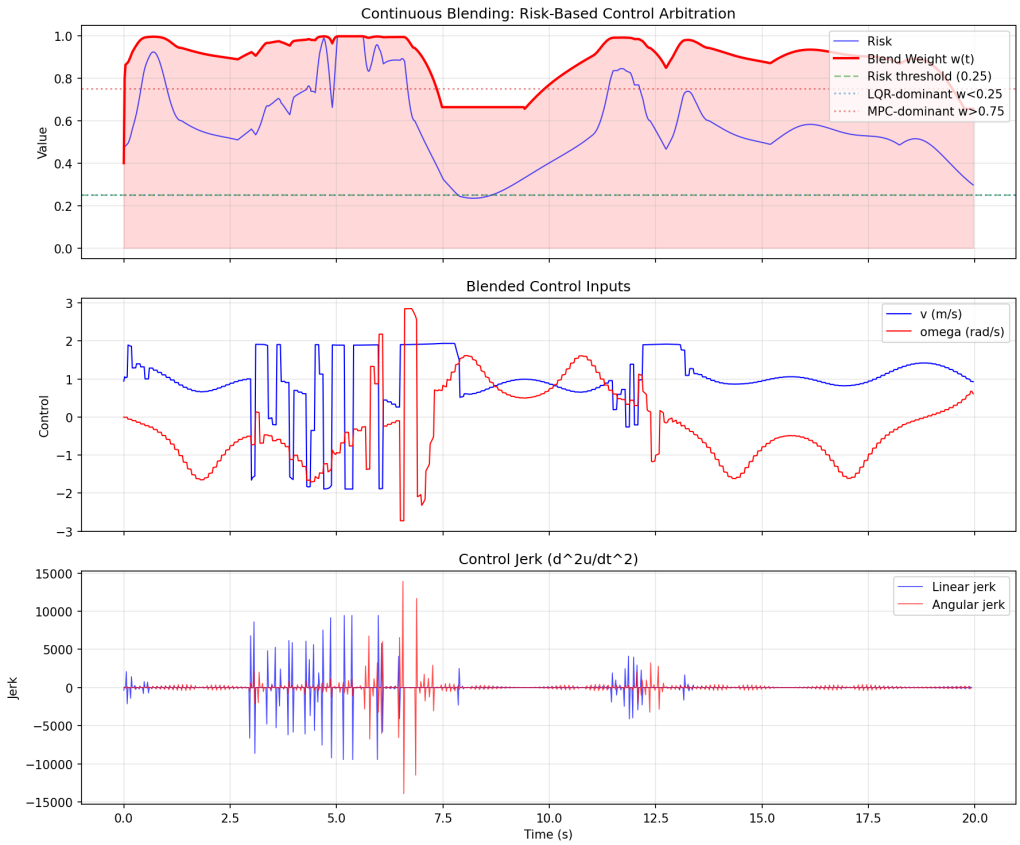

Figure 12: Blending weight $w(t)$ and risk level over time. The sigmoid-based supervisor smoothly transitions between LQR-dominant and MPC-dominant modes.

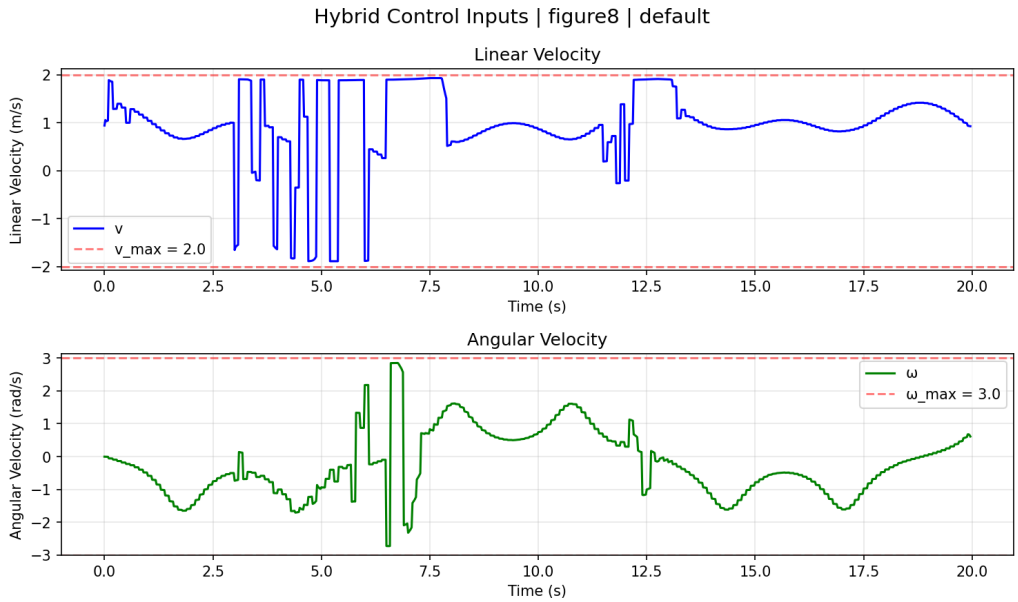

Figure 13: Hybrid control inputs showing smooth blending between LQR and MPC commands.

6 Comparative Summary

Table 1: Performance comparison across all four controllers on the figure-8 trajectory (20 seconds, $\Delta t = 0.02$s).

Metric	LQR	Tube MPC	Adaptive NMPC	Hybrid
Mean Error (m)	0.005	1.638	1.282	0.218
Final Error (m)	0.003	2.269	1.978	0.056
Mean Solve Time (ms)	<1	163	145	102
Obstacles	None	3	3	3
Collisions	N/A	163	355	0

Key observations:

- LQR is the fastest and most accurate but has *no* obstacle avoidance capability.
- Tube MPC provides obstacle awareness but the linearized constraints lead to conservative or sometimes insufficient avoidance (163 collision events).
- Adaptive NMPC achieves lower tracking error than Tube MPC due to exact nonlinear dynamics, but the more aggressive paths result in more near-misses (355 collisions).
- The Hybrid controller achieves the best of both worlds: near-LQR tracking accuracy in safe zones with effective obstacle avoidance, and zero recorded collisions in this scenario.

7 Debugging Journey: Failures and Fixes

This section honestly documents the critical failures we encountered and how we resolved them.

7.1 Failure 1: Velocity Clipping Caused Robot Stalling

Problem: All four controllers initially clipped the linear velocity to $v \in [0, v_{\max}]$, preventing any reverse motion. When the robot overshot a curve or needed to recover from a heading misalignment, the controller would command a negative v to back up. Because this was clipped to zero, the robot stalled permanently while the reference trajectory moved on, causing exponentially growing tracking error.

Fix: We changed the clipping to $v \in [-v_{\max}, v_{\max}]$ across all controller nodes, allowing the differential-drive robot to reverse naturally. This single change resolved the stalling in LQR, MPC, and Hybrid modes.

7.2 Failure 2: Hybrid Pure Pursuit Oscillation

Problem: After dodging an obstacle, the Hybrid controller's pure-pursuit logic searched for a target waypoint at a distance of 0.4m *from the robot*. Since the robot was already pushed > 0.4m away from the path, the algorithm selected the closest trajectory point, causing a harsh 90° turn. The robot would overshoot the path, triggering the same behavior in reverse, resulting in back-and-forth wandering.

Fix: We changed the pure-pursuit search to find a point 0.4m ahead *along the trajectory* starting from the nearest path point. This guarantees a smooth, forward-facing merge regardless of the robot's cross-track deviation.

7.3 Failure 3: MPC Cold-Start Infeasibility

Problem: The CVXPY-based Tube MPC failed to find a solution on the first few timesteps if the initial heading error was large. The linearized obstacle constraints were immediately violated, and without slack variables the problem was infeasible. The robot would not move at all.

Fix: We enabled soft constraints with slack variables ($\rho = 5000$) by default, allowing the solver to always return a usable solution even when hard constraints cannot be exactly satisfied.

7.4 Failure 4: RViz Reference Path Flickering

Problem: In the ROS2 deployment, the reference trajectory displayed in RViz would appear and disappear randomly. We were publishing the static path at 1 Hz with a continuously updating timestamp, causing RViz to clear and redraw the path every second with network-induced jitter.

Fix: We switched the reference path publisher to use the `TRANSIENT_LOCAL` QoS durability policy and published the path exactly once. The ROS middleware permanently stores the message, so any late-joining RViz subscriber immediately receives the complete path without flickering.

7.5 Failure 5: BlendingSupervisor API Mismatch

Problem: The `run_simulation.py` script passed `lqr_dominant_threshold` and `mpc_dominant_thresh` as constructor keyword arguments to `BlendingSupervisor`, but the class only accepts them as internal attributes.

Fix: We removed the invalid kwargs from the constructor call and set the attributes directly on the instance after construction.

8 ROS2 Implementation

The complete framework was deployed as a ROS2 Jazzy package running inside a Docker container. This ensures reproducibility and isolates the complex dependency chain (CVXPY, CasADi, IPOPT, Gazebo).

8.1 Architecture

The ROS2 workspace contains two packages:

- `hybrid_controller`: A pure Python package containing the core control algorithms (LQR, MPC, Adaptive MPC, Hybrid Blender, Risk Metrics).
- `hybrid_nav`: A ROS2 package containing the controller nodes, launch files, and RViz configuration for the Turtlebot3 Burger.

Each controller is wrapped as a ROS2 node that subscribes to `/odom` and `/obstacles`, computes the control output at 20 Hz, and publishes `TwistStamped` on `/cmd_vel`. Additionally, each node publishes:

- `/hybrid/reference_path` (TRANSIENT_LOCAL): The full reference trajectory for RViz.
- `/hybrid/robot_trail`: A continuously growing `nav_msgs/Path` for visualizing the robot's actual path.
- Diagnostic topics for tracking error, controller mode, and blending weight.

8.2 Logging

Every controller node writes a CSV-formatted telemetry log to `logs/` with columns: `Tick, X, Y, Theta, Error, V, Omega, Mode`. This enables post-run debugging of stalls, oscillations, and solver failures.

8.3 Simulation Video

A video demonstration of the ROS2 Gazebo simulation is available at:

YouTube: ROS2 Implementation

9 What Remains and Future Scope

9.1 Current Limitations

- **Collision count in standalone MPC/AMPC:** The Tube MPC and Adaptive NMPC still record collision events in the offline simulation. The linearized obstacle constraints (MPC) and the aggressive nonlinear paths (AMPC) need further tuning of d_{safe} and slack penalties.
- **Solver latency:** The mean MPC solve time of 100–160 ms is acceptable for 20 Hz control but would be insufficient for faster dynamics. Code-generated solvers (e.g., via `cvxpygen`) were partially implemented but not deployed.
- **Dynamic obstacles:** The current framework treats obstacles as static. Obstacle velocity estimation and predictive avoidance are not implemented.

9.2 Future Work

1. **Hardware deployment:** Port the Dockerized ROS2 nodes to a physical Turtlebot3 and replace pseudo-obstacles with real LiDAR-based clustering.
2. **Dynamic obstacle prediction:** Integrate a Kalman filter or constant-velocity model to predict obstacle trajectories, extending the MPC constraints into spatiotemporal tubes.
3. **C++ code generation:** Use CasADi's C code generation to achieve > 50 Hz control rates on embedded hardware.
4. **Multi-agent extension:** Extend the framework to multi-robot coordination with inter-robot collision avoidance.
5. **Formal safety guarantees:** Integrate Control Barrier Functions (CBFs) to provide provable safety certificates alongside the MPC constraints.

References

[1] B. Anderson and J. Moore, *Optimal Control: Linear Quadratic Methods*, Dover Publications, 2007.

[2] J. Rawlings, D. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed., Nob Hill Publishing, 2017.

[3] J. Köhler et al., "Certainty-Equivalent Adaptive MPC for Uncertain Nonlinear Systems," in *Proc. IEEE CDC*, 2026.

[4] J. A. E. Andersson et al., "CasADi: A Software Framework for Nonlinear Optimization and Optimal Control," *Mathematical Programming Computation*, vol. 11, pp. 1–36, 2019.

[5] S. Diamond and S. Boyd, "CVXPY: A Python-Embedded Modeling Language for Convex Optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[6] ROBOTIS, "TurtleBot3 e-Manual," https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/.

[7] Open Robotics, "ROS 2 Jazzy Jalisco Documentation," https://docs.ros.org/en/jazzy/.